

Organização de Cluster com Namespaces

O que você aprenderá nesta página

- Organizar recursos por namespace
- Separar ambientes e responsabilidades
- Aplicar comandos com contexto correto

Conforme a quantidade de aplicações cresce, colocar tudo no namespace default torna o cluster confuso. Namespaces são uma forma de separar recursos logicamente dentro do mesmo cluster. Eles não criam isolamento absoluto de segurança por si só, mas ajudam a organizar ambientes, equipes, aplicações e ferramentas de infraestrutura.

Um projeto pode usar namespaces como dev, staging e prod, ou separar por domínio, como observability, database e apps. O mais importante é que a separação reflita uma necessidade real de operação.

```
kubectl create namespace dev
kubectl get namespaces
kubectl get pods -n dev
```

Aplicando recursos em um namespace

Um recurso pode declarar o namespace no próprio manifesto ou receber o namespace no comando kubectl. Quando o namespace é declarado no YAML, o manifesto fica mais explícito. Quando ele é passado por linha de comando, o mesmo arquivo pode ser reutilizado em ambientes diferentes.

```
apiVersion: v1
kind: Namespace
metadata:
  name: apps
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-dep
  namespace: apps
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: nginx:1.27
```

Contextos do kubectl

Também é possível configurar o namespace padrão de um contexto. Isso reduz repetição, mas exige cuidado: muitos erros em clusters reais acontecem porque alguém aplicou manifestos no namespace errado.

```
kubectl config set-context --current --namespace=apps
kubectl config view --minify
```

Relação com configuração e acesso

Namespaces delimitam onde vivem ConfigMaps, Secrets, Services, Deployments, StatefulSets e ferramentas como Prometheus. Um Service chamado `api-svc` no namespace `apps` não é o mesmo que um Service de mesmo nome no namespace `dev`. De outro namespace, a aplicação deve chamar `api-svc.apps` ou o nome completo do DNS interno.

Boas práticas

Crie namespaces para separar responsabilidades reais. Use nomes simples, previsíveis e documentados. Em produção, combine namespaces com RBAC, quotas e políticas de rede.